



DONACIONES PÚBLICAS

INMUTABLES | DESCENTRALIZADAS

Plataforma de donaciones públicas e inmutables

Autor	Miguel Angel Gafe
Curso	Introducción a Ethereum y Solidity - ETH KIPU - TALENTO TECH
Track	Finanzas Descentralizadas (DeFi)

Repositorio

[https://github.com/miguelangelgafe/Introduccion-a-Ethereum-y-Solidity—ETH-KIPU](https://github.com/miguelangelgafe/Introduccion-a-Ethereum-y-Solidity-ETH-KIPU)

1. Definición del Problema

Las campañas de donación tradicionales (bancos, mercadopago, transferencias bancarias) operan bajo un modelo de **confianza ciega**. El donante transfiere dinero a una cuenta privada y no tiene ninguna garantía de que los fondos sean utilizados para el fin declarado. Esta opacidad genera múltiples ineficiencias:

- **Falta de transparencia total:** El organizador puede recaudar más dinero del declarado públicamente. No existe un libro contable auditable por terceros.
- **Desvío de fondos no detectable:** Sin un registro público de gastos, un mal actor puede utilizar el dinero para fines personales sin que los donantes puedan demostrarlo fehacientemente.
- **Desconfianza sistémica:** La proliferación de fraudes en campañas solidarias quita la voluntad de donar del público general.
- **Alta fricción y costos de auditoría:** Si un donante quiere verificar el uso de los fondos, debe contratar un contador o solicitar informes internos al organizador, lo cual es lento, costoso y no garantiza la veracidad.

2. Propuesta de Valor

La plataforma propuesta, resuelve estos problemas utilizando la infraestructura de Ethereum para crear un **libro contable público, inmutable y automatizado** de toda la vida financiera de una campaña.

- **Transparencia radical:** Cada donación (donante, monto, timestamp, mensaje) queda registrada públicamente en la blockchain. Cualquier persona puede ver el saldo actual del contrato en tiempo real.
- **Rendición de gastos forzosa:** El organizador (solo él) puede retirar fondos, pero cada retiro debe incluir una justificación obligatoria. El historial de retiros es público e inmutable.
- **Auditabilidad:** No se necesitan contadores ni intermediarios. La blockchain misma actúa como el verificador universal. Un donante puede consultar cuánto dinero entró, cuánto salió y en qué se gastó.
- **Automatización de reglas:** El contrato puede extenderse para incluir votaciones de donantes, aprobación múltiple de gastos (DAOs), o liberación escalonada de fondos por hitos.

¿Por qué blockchain y no una base de datos tradicional?

ACA iría una tabla comparativa, mas que nada, para poder ver rapidamente, las ventajas de usar la blockchain por sobre una BD tradicional(sistema centralizado)

3. Arquitectura del Sistema

La solución se compone de una única capa central (el Smart Contract en Ethereum).

Capa de Smart Contracts (Solidity)

Capa de Smart Contracts

El contrato inteligente actúa como una caja fuerte pública y auditable. No guarda datos redundantes ni hace cálculos pesados, solo registra donaciones y retiros de forma ordenada.

- **Estructuras (Structs)** Se define `Donacion` que guarda quién donó (`address`), cuánto (`uint256`), cuándo (`timestamp`) y un mensaje opcional (`string`). También se define `Retiro`, que guarda el monto retirado, la fecha y una justificación obligatoria
- **Mapeos (Mappings)** un mapping de `address => uint256` para llevar la cuenta de cuánto donó cada persona (más que nada para reconocimiento).
- **Modificadores de acceso (modifiers)** Se implementa `onlyOwner` (heredado de `OpenZeppelin`) para que solo el creador de la campaña pueda retirar fondos. También un modificador `esActiva` que impide donar después de que el organizador cierra la campaña.
- **Eventos (Events)** Se emiten `DonacionRecibida` y `FondoRetirado`. Esto permite que cualquier frontend (un explorador de bloques como Etherscan) muestre en tiempo real los movimientos sin tener que leer todo el estado del contrato.

Componentes Externos y Flujo de Datos El contrato solo maneja la lógica de donaciones y retiros. Para que sea usable por personas normales, se necesitan algunos componentes auxiliares:

- **Almacenamiento** Las facturas o comprobantes de gastos no se guardan directamente en la blockchain porque sería caro. solo guarda el hash en el campo `justificacion` del retiro. Cualquier donante puede usar ese hash para buscar el archivo(se usaria un ipfs o algun R2) y verificar que no este corrupto, modificado y por lo tanto verifica el gasto
- **Frontend web (dApp)** Una página HTML con Ethers.js que se conecta a MetaMask. Desde ahí los usuarios pueden:
 - Ver el saldo actual de la campaña.
 - Donar escribiendo un monto y un mensaje.
 - Consultar el historial de retiros con sus justificaciones. El frontend escucha los eventos del contrato para actualizarse solo.
 - No se usan oráculos en esta versión.

Interacción de Usuario (Flujo) El flujo es muy directo:

El organizador despliega el contrato desde su wallet (MetaMask) con el nombre de la campaña y una descripción. Automáticamente se convierte en el owner.

Cualquier donante entra a la página web, conecta su wallet, escribe un monto (por ejemplo 0.01 ETH) y un mensaje. Aprieta “Donar”, firma la transacción y

la donación queda registrada públicamente.

El organizador cuando necesita usar el dinero ejecuta “Retirar fondos”. El sistema le pide que pegue un link o un hash IPFS de la factura. Al firmar la transacción, el dinero se transfiere a su wallet y el retiro queda en el historial.

Los donantes pueden entrar en cualquier momento y ver cuánta plata hay, cuánto se retiró y para qué se gastó. Si ven una justificación rara (una factura falsa), pueden reclamar públicamente porque todo es inmutable.

4. Stack Tecnológico

Componente	Tecnología Seleccionada
Lenguaje de contratos	Solidity v0.8.20
Entorno de desarrollo	Foundry
Red de pruebas	Sepolia Testnet
Librerías de seguridad	OpenZeppelin (Ownable)
Almacenamiento de facturas	IPFS
Conexión Web3 / Frontend	Ethers.js + HTML/CSS
Wallet para pruebas	MetaMask (Sepolia)

5. Análisis de Riesgos

Para garantizar la viabilidad del proyecto en un entorno real (Mainnet), se han identificado los siguientes riesgos con sus respectivas mitigaciones:

- **Riesgo 1: Costo de gas elevado para donaciones pequeñas**

Descripción: Una donación de 1 USD puede costar 5 USD en gas si la red está congestionada.

Mitigación: El contrato está diseñado para funcionar en **capas 2 (Arbitrum, Optimism, Polygon)** donde el gas es significativamente menor (centavos de dólar). Despliegue en Polygon para donaciones de bajo valor.

- **Riesgo 2: Organizador malicioso que retira fondos con justificaciones falsas**

Descripción: El organizador puede poner cualquier string en **justificacion**, incluso una factura falsa. El contrato no valida el contenido.

Mitigación: La propuesta de valor no es “impedir el fraude”, sino “hacerlo detectable y público”. Cualquier donante puede ver que el organizador subió una justificación sospechosa y escracharlo públicamente. Además, se puede extender el contrato con un **mecanismo de aprobación por parte de los donantes** (votación simple) antes de liberar fondos grandes (DAO).

- **Riesgo 3: Pérdida de clave privada del organizador**

Descripción: Si el organizador pierde acceso a su wallet, los fondos quedan atrapados en el contrato para siempre.

Mitigación: El contrato no incluye una función de recuperación por diseño

(para evitar que un tercero robe). Se recomienda al organizador usar una wallet hardware o multisig (Gnosis Safe) para campañas con montos altos.

- **Riesgo 4: Ataque de reentrancia en retiro de fondos**

Descripción: Un contrato malicioso podría llamar recursivamente `retirarFondos` antes de que se actualice el balance.

Mitigación: Se sigue el patrón “Checks-Effects-Interactions”: primero se actualizan los registros (`retiros.push`, `totalRetirado`), *luego* se transfiere el dinero. Esto previene la reentrancia.

- **Riesgo 5: Donaciones múltiples desde cuentas anónimas (lavado de dinero o manipulación)**

Descripción: Un atacante puede crear muchas wallets sin identidad verificable y realizar pequeñas donaciones desde cada una para simular apoyo popular o mezclar fondos ilícitos (lavado de dinero). El organizador (coludido o engañado) luego retira el total, permitiendo blanquear el dinero.

Mitigación: Se establece un límite de donación por wallet sin KYC (ej. 0.5 ETH máximo por dirección). Para donaciones mayores o para el organizador, se requiere verificación de identidad (KYC) mediante un servicio verificación de identidades descentralizado.

6. Comparacion con plataformas existentes (Buscar plataformas que implementen la misma solucion y realizar un cuadro comparativo)

7. Conclusion

La integración de un smart contract en el ecosistema de Ethereum permite resolver el problema crónico de la opacidad y la falta de confianza en las donaciones solidarias. Al automatizar el registro de cada donación y cada retiro mediante reglas inmutables, se eliminan los intermediarios que podrían ocultar información y se asegura que el historial completo de la campaña quede grabado para siempre en la blockchain. Cualquier donante puede auditar en tiempo real el saldo disponible y verificar que cada gasto haya sido justificado públicamente. Este diseño demuestra que la tecnología Web3 ofrece una solución arquitectónicamente coherente (con un contrato simple pero robusto), económicamente viable (especialmente al desplegar en capas 2 como Polygon) y técnicamente accesible para organizaciones sociales, refugios de animales, comedores barriales y cualquier iniciativa que requiera transparencia radical en la gestión de fondos.

- **Transparencia radical:** Saldo y movimientos públicos 24/7.
- **Rendición forzosa:** El organizador no puede retirar sin dejar una justificación pública.
- **Auditabilidad cero-costos:** Un donante con un explorador de bloques (Etherscan) puede verificar todo en segundos.
- **Sin intermediarios extractivos:** No hay comisiones por plataforma; solo el costo de gas (mínimo en L2 como Polygon).